De la maquette à une représentation hiérarchique

Étude comparative de quatre approches de génération de code frontend

KOUONGA TADIE KENNEDY

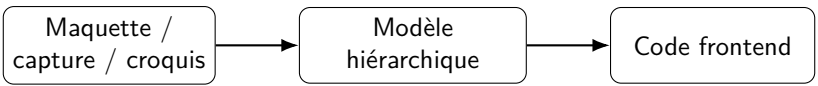
Universite de DOUALA

Jeudi 6 août 2026

Question de recherche

Problème étudié

Comment passer d'une représentation visuelle d'une interface (maquette, capture d'écran, croquis, etc.) à une **représentation structurée et hiérarchique** qui pourra ensuite servir à générer automatiquement du code frontend ?



Source : Problématique de recherche ; travaux étudiés

Pourquoi passer par une représentation intermédiaire ?

- Une image décrit surtout **ce que l'on voit**.
- Le code doit également exprimer **comment les éléments sont organisés**.
- Il faut donc retrouver :
  - les éléments de l'interface ;
  - leurs relations ;
  - leur organisation spatiale ;
  - et, selon la méthode, leurs comportements de mise en page.
- Les quatre travaux étudiés ne construisent cependant pas tous le même type de représentation.

Point important

Dans cette étude, « représentation hiérarchique » est utilisé au sens large : arbre de composants, arbre de mise en page, arbre de subdivisions ou structure de mise en page avec contraintes.

Les quatre approches étudiées

Approche	**Idée principale pour structurer l'interface**
Prototype2Code	Reconstruire une hiérarchie à partir des relations de contenance, puis organiser les éléments en lignes/colonnes.
DCGen	Diviser récursivement l'image en segments ; la division elle-même forme un arbre.
ScreenCoder	Détecter et nommer les grandes régions, puis construire un arbre de mise en page à partir de règles de conception web.
DesignCoder	Diviser l'interface, extraire le sens des éléments, puis les regrouper dans un arbre de composants.

Source : Synthèse des quatre documents fournis

1. Prototype2Code : partir des relations de contenance

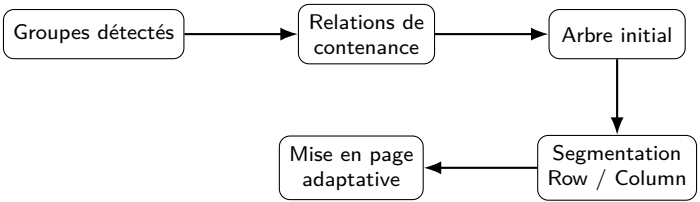
Entrée

Une maquette contenant des éléments visuels et des informations de conception.

1. Nettoyer les informations issues de la maquette.
2. Identifier les groupes et les éléments terminaux.
3. Déterminer quels éléments sont contenus dans quels groupes.
4. Construire un arbre de groupes imbriqués.
5. Organiser ensuite les enfants en **Row** ou **Column**.

Source : Document fourni sur Prototype2Code

Prototype2Code : construction du modèle



- La hiérarchie repose d'abord sur la **géométrie de la maquette**.
- Les éléments sont placés sous le plus petit groupe qui les contient.
- Les chevauchements peuvent conduire à un nœud utilisant un positionnement absolu.
- La seconde phase cherche une organisation compatible avec Flexbox.

Source : Document Prototype2Code fourni, section sur l'UI Layout Tree

2. DCGen : construire un arbre en divisant l'image

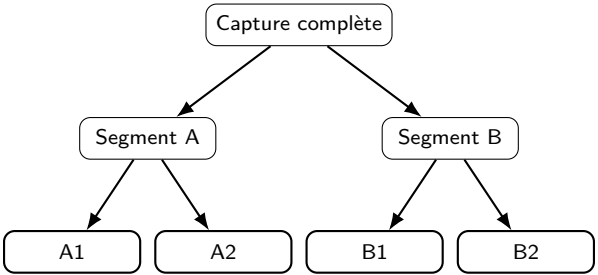
Idée centrale

Au lieu d'essayer de comprendre toute la page en une seule fois, DCGen cherche des **lignes de séparation** et découpe récursivement la capture en parties plus simples.

1. Détecter des séparations horizontales et verticales.
2. Découper l'image en segments.
3. Répéter le découpage sur les segments obtenus.
4. Stocker ces divisions sous forme d'un arbre.
5. Générer le code des petites parties puis les réassembler.

Source : Wan et al., 2025

DCGen : la hiérarchie représente les divisions



- Le nœud parent représente une zone avant sa subdivision.
- Les feuilles représentent les plus petites zones obtenues.
- L'arbre sert ensuite à reconstruire progressivement le code de la page.

Différence importante

Ici, l'arbre décrit d'abord une **décomposition spatiale de l'image**, et non une compréhension sémantique complète des composants.

3. ScreenCoder : détecter, planifier

Idée centrale

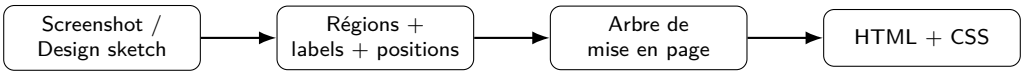
ScreenCoder décompose la génération d'interface en trois étapes : **comprendre visuellement**, **construire la structure**, puis **générer le code**.

1. **Grounding : comprendre l'interface avec un MLLM**
   Le **MLLM** analyse la capture d'écran pour identifier les regroupements visuels importants et leur attribuer un rôle : *header, sidebar, navigation, contenu, etc.*
   Pour chaque région détectée, il fournit une **bounding box** définie par :

$$(x, y, width, height)$$

2. **Planning : construire l'arbre de mise en page**
   Les régions détectées sont ensuite organisées à partir de leurs **positions et dimensions**.
   Les relations spatiales entre les bounding boxes permettent de déterminer la hiérarchie : quelles régions contiennent d'autres régions et quelles régions sont placées côte à côte.
   On obtient ainsi un **arbre de mise en page hiérarchique**, proche de la structure d'un DOM.

ScreenCoder : comment le modèle est construit ?



- Le planning convertit ces informations 2D en une structure de type DOM.
- Les coordonnées sont normalisées en pourcentage pour aider à préserver la structure.
- Les conteneurs internes peuvent être organisés avec des mécanismes de mise en page comme Grid.

Source : ScreenCoder, sections Grounding et Planning

4. DesignCoder : comprendre les composants avant l'arbre

Idée centrale

DesignCoder ne se limite pas à la position des éléments : il cherche à comprendre leur **rôle** et leur **contexte** avant de construire la hiérarchie des composants.

1. **Entrée : image + métadonnées**
   La maquette fournit une image ainsi que les métadonnées de chaque élément : [(id, type, bbox, style)]
   La *bbox* indique notamment la position et les dimensions de l'élément.
2. **UI Division : découper l'interface en régions**
   Le **MLLM analyse l'image et les métadonnées** pour diviser l'interface en sous-régions cohérentes.
   Une région correspond à un **ensemble d'éléments liés par leur rôle ou leur fonction**, par exemple une barre de recherche, une bannière ou une barre de navigation.
3. **Semantic Extraction : comprendre chaque élément**
   Le MLLM observe chaque élément dans son contexte visuel et décrit sa **fonction et son rôle**.

DesignCoder : du regroupement à l'arbre de composants

Component Grouping

Après avoir identifié les régions et compris le rôle des éléments, le MLLM regroupe les éléments selon plusieurs relations.

- **Relations spatiales** : position, proximité, alignement et organisation des éléments.
- **Relations sémantiques** : éléments ayant un rôle ou un sens commun.
- **Relations fonctionnelles** : éléments participant à une même fonctionnalité.

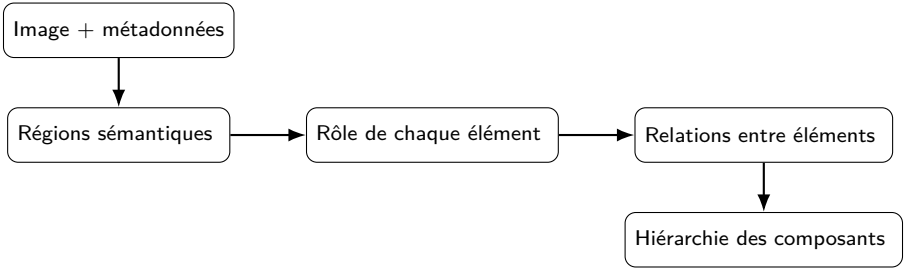
Construction de l'arbre

Le regroupement transforme la liste plate des éléments en une **hiérarchie de composants**. Les composants regroupés deviennent des **nœuds parents**, et les éléments qui les composent deviennent leurs **enfants**.

Position / bbox + contexte visuel + sémantique + fonction $\Rightarrow$ regroupement $\Rightarrow$ arbre

Source : Chen et al., DesignCoder

DesignCoder : comment les éléments deviennent un arbre ?



Ce qui détermine le regroupement

Position / bbox + **Sens visuel** + **Fonction** $\Longrightarrow$ **Composant**

- La **bbox** fournit la position et les dimensions des éléments.
- Le **MLLM** interprète leur rôle et leur contexte visuel.
- Les relations **spatiales, sémantiques et fonctionnelles** guident le regroupement.
- Le regroupement produit finalement un **arbre hiérarchique de composants**.

Source : DesignCoder, UI Grouping Chain

Comparaison : quelle information permet de construire la hiérarchie ?

Méthode	Principe de construction	Information dominante
Prototype2Code	Relations de contenance puis Row/Column	Géométrie + structure de la maquette
DCGen	Découpage récursif de l'image	Séparations et organisation spatiale
ScreenCoder	Détection de régions puis mapping vers un arbre	Régions nommées + positions + règles frontend
DesignCoder	Division + sémantique + regroupement	Sémantique + relations spatiales/fonctionnelles + métadonnées

Constat

Les méthodes cherchent toutes à retrouver une structure, mais **elles ne cherchent pas exactement la même structure**.

Comparaison : nature de la représentation obtenue

Méthode	Représentation intermédiaire	Rôle pour le code
Prototype2Code	Arbre de mise en page avec groupes et Row/Column	Guide l'imbrication HTML et les styles Flexbox/positionnement
DCGen	Arbre de subdivisions de l'image	Guide l'assemblage progressif du code
ScreenCoder	Arbre de layout de type DOM	arbre de composants pour HTML/CSS
DesignCoder	Arbre hiérarchique de composants enrichi par les styles	Guide la génération structurée des composants et du CSS

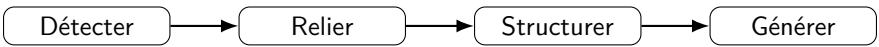
Source : Synthèse des documents étudiés

Comparaison : points forts et limites

Méthode	**Point fort pour notre problème**	**Limite / difficulté**
Prototype2Code	Modèle directement orienté vers la mise en page et le code	Dépend fortement des informations de conception et de règles géométriques
DCGen	Obtient une hiérarchie même à partir d'une capture d'écran	L'arbre représente surtout les divisions spatiales, pas nécessairement les composants sémantiques
ScreenCoder	Sépare clairement perception, structure et génération	Le planning repose sur des connaissances de mise en page web
DesignCoder	Combine géométrie, sémantique et hiérarchie de composants	Dépend d'un traitement multimodal et d'informations de design riches

Ce que les quatre méthodes ont en commun

1. **Identifier des unités de l'interface** : éléments, régions ou widgets.
2. **Déterminer des relations** entre ces unités.
3. **Construire une structure** permettant de ne plus traiter l'interface comme une simple image plate.
4. **Utiliser cette structure** pour faciliter la génération ou la reconstruction du code.



Ce qui différencie réellement les approches

- **Qu'est-ce qui est détecté ?**
  - éléments visuels ;
  - régions ;
  - groupes ;
  - widgets ;
  - ou simplement zones de l'image.
- **Comment la relation parent–enfant est-elle déterminée ?**
  - contenance géométrique ;
  - découpage spatial ;
  - raisonnement sémantique ;
  - règles de mise en page ;
- **Que doit représenter le modèle ?**
  - la structure visuelle ;
  - les composants ;
  - la mise en page ;
  - ou plusieurs de ces aspects.

Lecture des méthodes par rapport à notre question

Notre objectif

Obtenir une représentation intermédiaire suffisamment riche pour permettre ensuite de générer du code frontend.

- **Prototype2Code** montre l'intérêt d'un arbre orienté mise en page.
- **DCGen** montre qu'une hiérarchie peut émerger directement d'une décomposition de l'image.
- **ScreenCoder** montre l'intérêt de séparer **détection** et **construction de la structure**.
- **DesignCoder** montre l'intérêt d'ajouter la **sémantique** aux informations géométriques pour construire une hiérarchie de composants.

Question qui se dégage

Quel niveau d'information doit contenir notre modèle pour être indépendant de la technologie cible tout en restant suffisamment riche pour générer ensuite React, HTML/CSS, Flutter, etc. ?

Conclusion

Conclusion de l'étude

Les travaux étudiés convergent vers une même idée : **la maquette doit être transformée en une représentation structurée avant de produire le code**.

Mais plusieurs stratégies existent :

- **géométrique** : reconstruire les relations de contenance ;
- **spatiale** : découper récursivement l'image ;
- **structurelle** : convertir les régions en arbre de layout ;
- **sémantique** : comprendre les rôles et relations des composants.

La problématique devient alors : quelle combinaison de ces informations est nécessaire pour définir un modèle intermédiaire adapté à la génération multi-technologies ?

Références principales



S. Xiao, Y. Chen, J. Li, L. Chen, T. Zhou.
Prototype2Code : End-to-end Front-end Code Generation from UI Design Prototypes.
arXiv :2405.04975, 2024.
https://arxiv.org/abs/2405.04975



Y. Wan, C. Wang, Y. Dong, W. Wang, S. Li, Y. Huo, M. Lyu.
Divide-and-Conquer : Generating UI Code from Screenshots.
Proc. ACM Software Engineering, FSE, 2025.
https://doi.org/10.1145/3729364



Y. Jiang, Y. Zheng, Y. Wan, J. Han, Q. Wang, M. R. Lyu, X. Yue.
ScreenCoder : Advancing Visual-to-Code Generation for Front-End Automation via Modular Multimodal Agents.
arXiv :2507.22827, 2025.
https://arxiv.org/abs/2507.22827



Y. Chen, S. Ding, Y. Zhang, W. Chen, J. Du, L. Sun, L. Chen.
DesignCoder : Hierarchy-Aware and Self-Correcting UI Code Generation with Large Language Models.
arXiv :2506.13663, 2025.
https://arxiv.org/abs/2506.13663